

INGENIERÍA DE SOFTWARE

Unidad 03: Procesos del Software y Ciclos de Vida

El significado de proceso · Modelos de proceso · Iteración de procesos

Actividades fundamentales · Herramientas de modelado · Ayuda automatizada

Bibliografía base: Pressman, Ingeniería del Software (6.ª ed.) · Sommerville, Ingeniería de Software

Contenido de la clase

01 El significado de proceso

02 Modelos de proceso

03 Iteración de procesos

04 Especificación, diseño, implementación, validación y evolución

05 Herramientas y técnicas para modelado de procesos

06 Ayuda automatizada al proceso

TEMA 1

El significado de proceso

Qué es un proceso de software y cuál es su marco de trabajo genérico

¿Qué es un proceso de software?

Un proceso de software es un conjunto de actividades, acciones y tareas que se llevan a cabo para transformar una necesidad del usuario en un producto de software.

Tres conceptos que conviene distinguir

Proceso

El marco general de actividades a realizar (el "qué" y en qué orden).

Metodología

Un conjunto específico de métodos y prácticas para llevar a cabo el proceso (el "cómo").

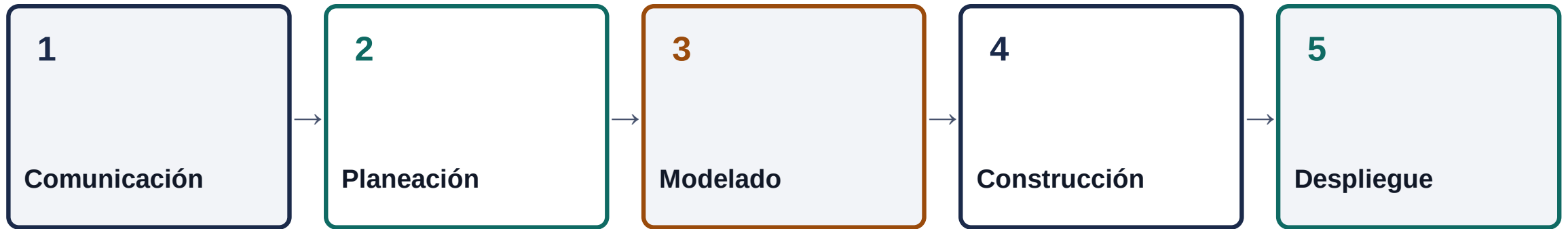
Modelo de proceso

Una representación abstracta de un proceso, usada para compararlo o elegirlo (cascada, incremental, etc.).

Fuente: Pressman, Cap. 2 — El proceso: una visión general, pp. 22–47.

El marco de trabajo genérico del proceso

Pressman propone cinco actividades genéricas, presentes en todo proceso de software:



Actividades "de la sombrilla"

Se realizan a lo largo de todo el proceso, cubriéndolo como un paraguas: gestión de proyectos, gestión de riesgos, aseguramiento de la calidad, gestión de configuración, revisiones técnicas y medición, entre otras.

Fuente: Pressman, Cap. 2, pp. 22–47.

TEMA 2

Modelos de proceso

Distintas formas de organizar las actividades del proceso de software

Modelo en cascada



Enfoque secuencial y lineal: cada actividad debe completarse antes de iniciar la siguiente. Es el modelo más antiguo y aún útil cuando los requisitos están bien definidos desde el principio.

Fuente: Pressman, Cap. 3 — Modelos prescriptivos de proceso, pp. 48–76.

Modelos incrementales

Combinan elementos secuenciales con elementos iterativos: cada incremento entrega una porción funcional del producto, cada vez más completa.

Incremento 1 — funcionalidad básica

Incremento 2 — funcionalidad ampliada

Incremento 3 — funcionalidad completa

Cada incremento es entregable y utilizable por el cliente, aunque no tenga todavía toda la funcionalidad final.

Fuente: Pressman, Cap. 3, pp. 48–76.

Modelos evolutivos: prototipos y espiral

Prototipos

Se construye una versión parcial y rápida del sistema para que el cliente visualice y valide requisitos poco claros, antes de construir el producto final.

Útil cuando: el cliente conoce los objetivos generales, pero no puede detallar entradas, procesos o salidas.

Modelo en espiral

Combina la naturaleza iterativa del prototipado con el control del modelo en cascada, agregando análisis de riesgo explícito en cada vuelta de la espiral.

Útil cuando: el proyecto tiene alto riesgo y se necesita evaluarlo en cada iteración antes de seguir invirtiendo.

Fuente: Pressman, Cap. 3, pp. 48–76.

Modelos concurrentes

Representan las actividades del proceso ocurriendo en paralelo, no en secuencia estricta: mientras se modela una parte del sistema, otra ya puede estar en construcción.



Un mismo elemento del sistema puede estar en distintos estados al mismo tiempo que otros elementos.

Fuente: Pressman, Cap. 3, pp. 48–76.

TEMA 3

Iteración de procesos

Por qué (casi) todo proceso moderno vuelve una y otra vez sobre lo ya hecho

¿Por qué iterar?

Los requisitos cambian, el cliente aprende sobre lo que necesita a medida que ve resultados, y los riesgos se reducen si se revisan avances parciales en lugar de esperar al final.

Cambio

Los requisitos evolucionan durante el proyecto, no solo al principio.

Aprendizaje

Ver resultados parciales ayuda al cliente a precisar lo que realmente necesita.

Riesgo

Detectar un problema en una iteración temprana es mucho más barato que al final.

Los modelos incrementales, evolutivos y concurrentes vistos antes son, en el fondo, distintas formas de organizar esa iteración. Las metodologías ágiles (más adelante en la materia) la llevan al extremo.

Fuente: Pressman, Cap. 3, pp. 48–76.

TEMA 4

Especificación, diseño, implementación, validación y evolución

Las cuatro actividades fundamentales de todo proceso de software, según Sommerville

Las actividades fundamentales del proceso

Sommerville identifica cuatro actividades presentes en cualquier proceso de software:

1

Especificación

Definir qué debe hacer el sistema y las restricciones sobre su operación.

2

Diseño e implementación

Producir el sistema que cumple con la especificación.

3

Validación

Comprobar que el software hace lo que el cliente quiere.

4

Evolución

Modificar el software en respuesta a necesidades cambiantes del cliente o del mercado.

¿Te suena parecido al "ciclo del software" de la Unidad 1? No es casualidad: son dos formas de nombrar prácticamente las mismas fases.

Fuente: Sommerville, Ingeniería de Software, Cap. 4 — Procesos del software.

TEMA 5

Herramientas y técnicas para modelado de procesos

Cómo representar el proceso en sí mismo, no el sistema que se construye

Representar el proceso, no el sistema

Así como en la Unidad 2 modelamos sistemas, acá el objeto a modelar es el propio proceso de trabajo: quién hace qué, en qué orden y con qué entregables.

Diagramas de flujo de proceso

Muestran la secuencia de actividades y las decisiones que la alteran.

Patrones de proceso

Describen un problema recurrente del proceso y una solución probada, reutilizable en distintos proyectos.

Diagramas de actividad (UML)

Notación formal para representar flujos de trabajo, con puntos de decisión y paralelismo.

Fuente: Pressman, Cap. 2, pp. 22–47.

TEMA 6

Ayuda automatizada al proceso

Herramientas que asisten y automatizan actividades del proceso de software

Herramientas CASE

CASE (Computer-Aided Software Engineering): herramientas de software que automatizan o asisten una o más actividades del proceso de desarrollo.

Upper-CASE

Apoyan las primeras etapas: análisis, diseño y modelado (ej. herramientas de diagramación UML).

Lower-CASE

Apoyan las etapas finales: generación de código, pruebas y depuración.

I-CASE (integradas)

Cubren el proceso completo, de punta a punta, en un mismo entorno.

Ejemplos actuales: herramientas de gestión de tareas y tableros (Jira, Trello), control de versiones (Git), integración continua, gestión de requisitos, y asistentes de código con IA.

Fuente: Pressman, Cap. 2, pp. 22–47.

Fuentes bibliográficas

Pressman, Roger S.

Ingeniería del Software: Un Enfoque Práctico (6.^a edición)

Capítulo 2 — "El proceso: una visión general", pp. 22–47, y Capítulo 3 — "Modelos prescriptivos de proceso", pp. 48–76.
Fuente principal de esta unidad.

Sommerville, Ian

Ingeniería de Software

Capítulo 4 — "Procesos del software" (numeración de ediciones recientes; puede variar). Fuente principal para las cuatro actividades fundamentales (Tema 4).

En resumen

No existe un único proceso correcto: existen distintos modelos, y cada proyecto exige elegir el más adecuado.

- Un proceso de software organiza las actividades genéricas (comunicación, planeación, modelado, construcción, despliegue) más las actividades de la sombrilla.
- Los modelos de proceso (cascada, incremental, evolutivos, concurrentes) son formas distintas de organizar esas actividades, cada uno con sus propios costos y beneficios.
- La iteración, la especificación/diseño/implementación/validación/evolución, y las herramientas CASE atraviesan a todos los modelos por igual.

Próximo paso: ejercitario de la Unidad 03